

VehicleSalesFIAP

Tech Challenge FIAP/SOAT - Fase 3

API para plataforma de revenda de veiculos

Repositorio	https://github.com/elianairparronchi/VehicleSalesFIAP
Video	PENDENTE - substituir pelo link do video apos gravacao e publicacao
Imagem GHCR	ghcr.io/elianairparronchi/vehiclesalesfiap

Observacao: antes do envio oficial, substitua o campo do video pelo link publicado.

Links Da Entrega

Repositorio GitHub:

<https://github.com/elienairparronchi/VehicleSalesFIAP>

Video demonstrativo:

PENDENTE - substituir pelo link do video apos gravacao e publicacao.

Imagem Docker publicada pelo CI/CD:

ghcr.io/elienairparronchi/vehiclesalesfiap

Resumo Da Solucao

O projeto implementa uma API para uma plataforma de revenda de veiculos. A solucao permite cadastrar veiculos para venda, editar veiculos disponiveis, listar veiculos a venda por preco crescente, realizar compra por comprador autenticado e listar veiculos vendidos tambem por preco crescente.

A autenticacao e o cadastro de compradores foram mantidos fora da API transacional, usando Keycloak em um container separado. A API recebe o token JWT emitido pelo Keycloak e persiste apenas o identificador externo do comprador no registro da venda.

Tecnologias

- .NET 10
- ASP.NET Core
- Clean Architecture
- Entity Framework Core
- SQL Server via Docker Compose
- Keycloak via Docker Compose
- Swagger/OpenAPI
- xUnit
- GitHub Actions
- GitHub Container Registry

Arquitetura

O projeto segue Clean Architecture:

- `VehicleSalesFIAP.Domain`: entidades, value objects e regras de negocio.
- `VehicleSalesFIAP.Application`: casos de uso, contratos e DTOs.
- `VehicleSalesFIAP.Infrastructure`: EF Core, SQL Server e implementacoes tecnicas.
- `VehicleSalesFIAP.Api`: controllers, autenticacao, autorizacao, Swagger e middleware.
- `VehicleSalesFIAP.Tests`: testes unitarios, integracao e fluxo fim-a-fim.

Essa separacao segue os principios estudados na pos-graduacao: regras de negocio isoladas de frameworks, inversao de dependencia, responsabilidades bem delimitadas e adaptadores externos para banco, HTTP e identidade.

Funcionalidades Entregues

Requisito	Implementacao
Cadastrar veiculo para venda	POST /api/v1/vehicles
Editar veiculo	PUT /api/v1/vehicles/{id}
Listar veiculos a venda por preco crescente	GET /api/v1/vehicles/available
Comprar veiculo por comprador cadastrado	POST /api/v1/vehicles/{id}/purchase
Listar veiculos vendidos por preco crescente	GET /api/v1/vehicles/sold
Separar cadastro/autorizacao de compradores	Keycloak separado da API e do banco transacional
Deploy automatizado	GitHub Actions com build e publicacao da imagem no GHCR

Como Executar Localmente

Pre-requisitos:

- .NET SDK 10
- Docker Desktop
- Git

Comandos principais:

```
git clone https://github.com/elianairparronchi/VehicleSalesFIAP.git
cd VehicleSalesFIAP
dotnet restore VehicleSalesFIAP.slnx
dotnet build VehicleSalesFIAP.slnx
dotnet test VehicleSalesFIAP.slnx
docker compose up -d --build
```

Servicos locais:

- API: <http://localhost:5000>
- Swagger: <http://localhost:5000/swagger>
- Health check: <http://localhost:5000/health>
- Keycloak: <http://localhost:8081>
- SQL Server: localhost,1433

Usuarios locais de teste:

Perfil	Usuario	Senha
Gestor	vehicle.manager	VehicleManager123!
Comprador	buyer.user	Buyer123!

Testes

A suite cobre regras de dominio, casos de uso, autorizacao e fluxo HTTP fim-a-fim.

Comandos usados na validacao:

```
dotnet build VehicleSalesFIAP.slnx --configuration Release
dotnet test VehicleSalesFIAP.slnx --configuration Release
dotnet format VehicleSalesFIAP.slnx --verify-no-changes --verbosity minimal
dotnet list VehicleSalesFIAP.slnx package --vulnerable --include-transitive
docker compose config
docker compose build api
```

Fluxo fim-a-fim validado:

1. Gestor autentica no Keycloak.
2. Gestor cadastra um veiculo.
3. API lista veiculos disponiveis por preco.
4. Comprador autentica no Keycloak.
5. Comprador efetiva a compra.
6. API registra a venda e marca o veiculo como vendido.
7. Gestor lista veiculos vendidos por preco.
8. API bloqueia uma segunda compra do mesmo veiculo.

CI/CD

O workflow `.github/workflows/ci.yml` executa em Pull Requests, pushes para `main`, tags `v*.*.*` e execucao manual.

Validacoes do pipeline:

- restore;
- formatacao com `dotnet format`;
- build em Release;
- testes com resultado `.trx`;
- cobertura com `XPlat Code Coverage`;
- verificacao de pacotes vulneraveis;
- build Docker;
- publicacao da imagem da API no GitHub Container Registry fora de Pull Requests.

O projeto tambem possui Dependabot para NuGet, GitHub Actions e Docker.

Evidencias Para O Video

O video demonstrativo deve mostrar:

- repositorio GitHub e estrutura da solucao;
- workflow de CI/CD;
- Docker Compose subindo API, SQL Server, Keycloak e migrations;
- Keycloak com comprador cadastrado no realm `vehiclesalesfiap`;
- Swagger da API;
- token do gestor;
- cadastro de veiculo;
- listagem de veiculos disponiveis;
- token do comprador;
- compra do veiculo;
- consulta mostrando status `Sold`;
- listagem de vendidos;
- bloqueio de segunda compra.

Observacao Antes Da Entrega Oficial

Antes de enviar este PDF para a FIAP, substitua o campo Video demonstrativo pelo link publico ou compartilhavel do video publicado.

Checklist Antes Do Envio

- Confirmar que o repositorio GitHub esta publico ou compartilhavel para avaliacao.
- Confirmar que o link do video foi publicado e esta acessivel.
- Substituir o texto pendente do video neste documento e regenerar o PDF.
- Conferir que o README possui instrucoes de execucao local e testes.
- Conferir que o workflow do GitHub Actions aparece executado no repositorio remoto.
- Conferir que o video mostra infraestrutura, cadastro/autenticacao do comprador, cadastro do veiculo, compra e efetivacao da venda.